Decision, computation & Language

课程内容

1. Finite state automation（有限状态自动化）
2. Regular expression（正规表达式）
3. Context-free grammar（上下文无关表达法）
4. Pushdown Automaton（叠加自动机）
5. Turing machine（图灵机）

主要概念：

特征	形式语言	非形式语言
定义方式	由精确、数学化的规则明确定义	由习惯、文化自然演化，规则模糊且灵活
核心目标	被机器无歧义地解析和执行	方便人类之间的沟通和交流
歧义性	基本无歧义，一句一义	充满歧义，依赖上下文理解
例子	Python, SQL, JSON, HTML, 二进制指令	中文，英语，注释，诗歌，日常对话
严格性	极其严格，违反规则会导致失败	非常灵活，可以容忍错误和不规范
变化	相对稳定变化需要通过更新标准或版本来完成	持续不断变化，会产生新的俚语和用法

1. Languages

Category：

Formal language

形式语言
定义：
形式语言是被精确定义的语言，其语法和语义由一组严格的、数学化的规则所支配。这些规则是明确的、无歧义的，并且通常与任何特定的含义或上下文分离。形式语言是为机器（如计算机）理解和处理而设计的。

核心特征：

精确性：语法规则被明确定义，通常使用形式文法（如巴科斯-诺尔范式 BNF）或自动机理论来描述。

无歧义性：一个符合语法的句子或表达式必须有且只有一种“正确”的解释（语义）。

上下文无关性：语句的含义不依赖于上下文（在理想情况下），只由其结构决定。

人造的：它们是由人类为了特定目的而故意设计和创造的。

Examples

1. Programming language;
2. Database query languages;
3. Various file format

Informal language

Examples

1. spoken languages;
2. comments
3. Informal Description:

非形式语言
定义：
非形式语言是自然演化而来的语言，其规则并非由一套严格的数学规范定义，而是通过习惯、文化和上下文来约定俗成。它们充满了模糊性、歧义性和灵活性，是为人类之间的交流而设计的。

Eg: a42 is a variable name, but 42a is not because a variable name can't start with a number.

The variable-name description is a combination of English and examples

Symbol:

字母表

Alphabet – a finite, nonempty set Σ of symbols.

一个字母表是一个有限的、非空的符号集合。通常用大写希腊字母 ∑ 或 Σ 来表示

String (word) – a finite sequence of symbols from the alphabet

字符串

一个字符串是由字母表中的符号构成的有限序列

Example

Σ = {*a, b*}, then *abab, aaaabbba* are strings on Σ

w = abaaa indicates the string named *w* has the specific value *abaaa*

Languages

empty strin

Empty string (word) – the string with no symbols, denoted as ϵ.

$$|\epsilon| = 0$$

$$\epsilon w = w\epsilon = w$$

Length of a string – the number of symbols in the string, denoted as $|w|$.

$$|w_1 w_2| = |w_1| + |w_2|$$

Reverse of a string – obtained by writing the symbols in reverse order, denoted as w^R

$$w = a_1\ a_2 \ldots\ a_n$$

$$w^R = a_n\ \ldots\ a_2\ a_1$$

A **palindrome** is a string w satisfying $w = w^R$.

eg: w = aba

Languages

The **concatenation** of two strings is obtained by appending the symbols to the right end (concatenation is associative).

. 连接（Concatenation）
定义：两个字符串的连接是通过将一个字符串的符号序列附加到另一个字符串的右端，形成一个新的字符串。连接操作是结合的（associative），这意味着对于任意三个字符串

$$w = a_1\ a_2 \ldots\ a_n$$

$$v = b_1\ b_2 \ldots\ b_n$$

$$wv = a_1\ a_2 \ldots\ a_n\ b_1\ b_2 \ldots\ b_n$$

前缀 后缀

If u, v, w are strings and $w=uv$ then u is a **prefix** of w and v is a **suffix** of w. A proper prefix of w is a prefix that is not equal to ϵ of w. (similarly for proper suffix)

ϵ不能作为 prefix 和 suffi

w^n denotes the concatenation of n copies of w

星闭包）：表示字母表 ∑ 上所有可能字符串的集合，包括空字符串

$\sum^*$ (**star closure** of $\sum$) denotes the set of all strings (words) over $\sum$.

表示字母表 ∑ 上所有非空字符串的集合

$\sum^+$ (**positive closure** of $\sum$) denotes the set of all non-empty strings (words) over $\sum$.

Both of $\sum^*$ and $\sum^+$ are infinite set. {a, ab, b, ba, ... }

Definition

Formal language

简单来说就是通过数学公式(rules)构建的语言 its is define by its form(syntax语法） not its meaning

A **formal language** is a set of strings constructed from a finite alphabet according to specific syntactical rules. It is used in various fields, such as computer science, linguistics, and mathematics, to define and analyze the structure of languages systematically.

A **language** L (or formal language) over alphabet $\sum$ is a subset of $\sum^*$.

We can express new languages in terms of other languages using concatenation and closure:

show fomal language can be constructed by mathmatical rules

$$L_1 L_2 = \{w_1 w_2 : w_1 \in L_1\ and\ \ w_2 \in L_2\}$$

$$L^* = \{w_1 w_2 \ldots w_n :\ n\ \geq 0\ \ and\ \ w_1, w_2, \ldots w_n \in L\}$$

2. Grammars

set of mathmatical rul

Applications in programming languages (stages of compilation):

1. **Lexical analysis**
2. **Parsing**
3. **Code generation**
4. **Code optimization**

字段

Lexical Analysis(词法分析): divide sequence of characters into tokens, such as variable names, operators, labels. In a natural language tokens are strings of consecutive letters (easy to recognize!)

Parse Tree(解析树): identify relationships between tokens.

Parse tree or parsing tree or derivation tree or concrete syntax tree is an ordered, rooted tree that represents the syntactic structure of a string according to some context-free grammar.

Code generation: is part of the process chain of a compiler and converts intermediate representation of source code into a form (e.g., machine code) that can be readily executed by the target system.

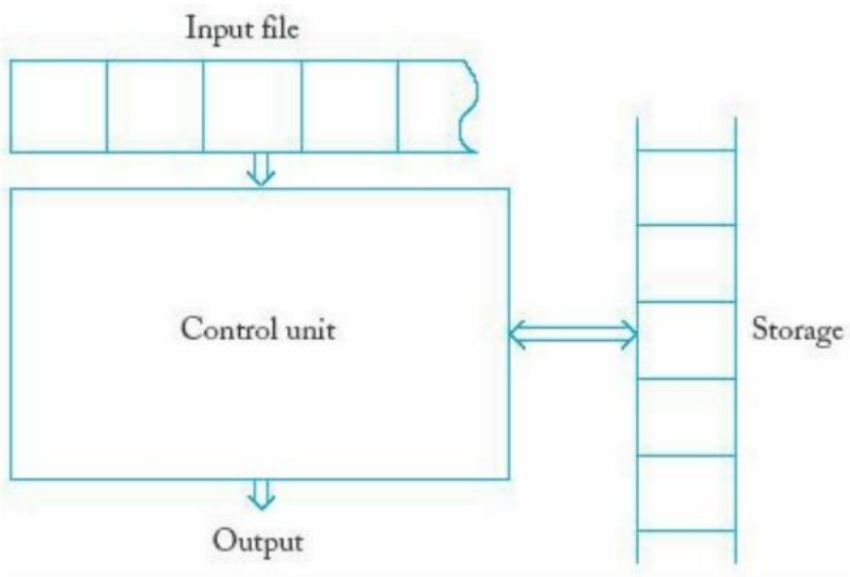
Code optimization: is the process of modifying a software system to make some aspect of it work more efficiently or use fewer resources. In general, a computer program may be optimized so that it executes more rapidly, or to make it capable of operating with less memory storage or other resources, or draw less power.

3. Automata

Automata

process the input file, control the storage and output something.

An automaton is an abstract model of a digital computer



1. **输入文件（Input file）：**

输入文件是自动机接收数据的起点。这些数据进入自动机，供接下来的处理使用。

2. **控制单元（Control unit）：**

控制单元是自动机的核心部分，它根据输入的数据执行预定义的操作。控制单元包含了处理逻辑，能够接收输入并控制存储器中的状态改变。

3. **存储器（Storage）：**

存储器用于保存临时数据或状态信息。控制单元可以读取或写入存储器，以调整和存储处理过程中产生的中间状态或结果。

4. **输出（Output）：**

输出是处理完输入数据后的结果，经过控制单元处理和可能的存储后，最终作为输出提供出来。

简而言之，自动机接收输入，通过控制单元的计算和存储单元的辅助处理，产生相应的输出。这是一个标准的输入-处理-输出模型，用以模拟数字计算机的基本工作原理。

4. Mathematical preliminaries

Complexity theory

Hard or Easy

On the other hand, a problem is called “hard”, if it cannot be solved efficiently, or if we don’t know whether it can be solved efficiently.

Computation theory

Solvable or Unsolvable.

Automata theory

Same power or not

more powerful means could accept more langauge or complex language.

Central question in Automata Theory: Do these models have the **same power**, or can one model solve more problems than the other?

Examples:

如果两个模型能解决完全相同的一类问题，我们就说它们是 “计算等价” 的，拥有相同的计算能力。
如果模型A能解决所有模型B能解决的问题，并且还能解决更多模型B无法解决的问题，我们就说模型A比模型B更强大。

如何比较？—

1. Finite Automata
2. Context-Free Grammars
   上下文无关文法 / 下推自动机
3. Turing Machines

Mathematical preliminaries

4.1 Set

A **set** is a collection of well-defined objects

Subset

If A and B are sets, then A is a **subset** of B, written as $A \subseteq B$, if every element of A is also an element of B.

(一个含有n个元素的集合含有2^n个子集)

Power set

If B is a set, then the **power set** P(B) of B is defined to be the set of all subsets of B: $P(B) = \{A : A \subseteq B\}$. Observe that $\varnothing \in P(B)$ and $B \in P(B)$.

Set

If A and B are two sets, then

- their **union** is defined as $A \cup B = \{x : x \in A \text{ or } x \in B\}$,
- their **intersection** is defined as $A \cap B = \{x : x \in A \text{ and } x \in B\}$,
- their **difference** is defined as $A \setminus B = \{x : x \in A \text{ and } x \notin B\}$,
- the **Cartesian product** of A and B is defined as $A \times B = \{(x, y) : x \in A \text{ and } y \in B\}$,
- the **complement** of A is defined as $A = \{x : x \notin A\}$.

4.2 Relation and function

A **binary relation** on two sets A and B is a subset of $A \times B$ (Cartesian product of A and B).

A **function** f from A to B, denoted by $f : A \to B$, is a binary relation R, having the property that for each element $a \in A$, there is exactly one ordered pair in R, whose first component is a. We will also say that $f(a) = b$, or f maps a to b, or the image of a under f is b. The set A is called the domain of f, and the set

$$\{b \in B : \text{there is an } a \in A \text{ with } f(a) = b\}$$

is called the range of f.

Injective, bijective and surjective

A function $f : A \rightarrow B$ is **one-to-one** (or **injective**), if for any two distinct elements a and a′ in A, we have $f(a) \neq f(a')$. The function f is **onto** (or **surjective**), if for each element $b \in B$, there exists an element $a \in A$, such that $f(a) = b$; in other words, the range of f is equal to the set B. A function f is a **bijection**, if f is both injective and surjective.

Equivalence Relation

A binary relation $R \subseteq A \times A$ is an **equivalence relation**, if it satisfies the following three conditions:

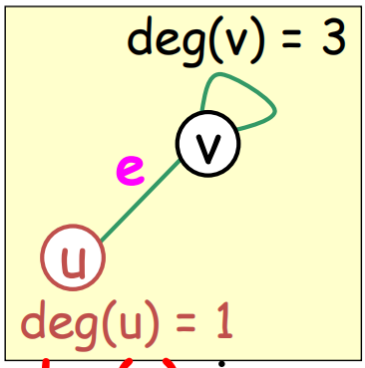
- R is **reflexive**: For every element in $a \in A$, we have $(a, a) \in R$.
- R is **symmetric**: For all a and b in A, if $(a, b) \in R$, then also $(b, a) \in R$.
- R is **transitive**: For all a, b, and c in A, if $(a, b) \in R$ and $(b, c) \in R$, then also $(a, c) \in R$.

4.3 Graph

Undirected graphs

In an undirected graph G, suppose that e = {u, v} is an edge of G

- u and v are said to be ***adjacent*** and called ***neighbors*** of each other.
- u and v are called ***endpoints*** of e.
- e is said to be ***incident*** with u and v.
- e is said to ***connect*** u and v.



- The ***degree*** of a vertex v, denoted by **deg(v)**, is the number of edges incident with it (a loop contributes twice to the degree)

Graph

The **degree** of a vertex v, denoted by deg(v), is defined to be the number of edges that are incident on v.

顶点之间不相互连接是完全允许的

A **path** in a graph is a sequence of vertices that are connected by edges. A path is a **cycle**, if it starts and ends at the same vertex. A **simple path** is a path without any repeated vertices. A graph is **connected**, if there is a path between every pair of vertices.

Simple Path

路径是连通的：序列中任意两个连续的顶点在图中的对应边是存在的。

顶点不重复：该路径上的所有顶点都是互不相同的（除了允许起点和终点是同一个顶点的情况，这种情况是"简单回路"，是简单路径的一种特殊形式）

边通常也不重复：由于顶点不重复，连接它们的边自然也不会重复（除非图包含自环，但自环会破坏顶点的"不重复"性，所以简单路径通常也不包含自环）

4.4 Proof techniques

1. Direct proof
2. Constructive proof 给出example
3. Non-constructive proof

构造性 = "我直接给你一个例子 / 算法，你可以把它写出来并验证"。

非构造性 = "我能证明某个例子一定存在，但我不（或不能）把它写出来"。

4. Contradiction proof

反证法
核心思想： 为了证明命题 P 为真，我们先假设其否定命题 ¬P 为真。然后通过逻辑推理，从这个假设出发，推导出一个矛盾（比如一个与已知公理、定理或假设 ¬P 不然为真。

5. Induction proof

Regular Graph（正则图）

图中每个顶点的度（Degree）都相等。

那么它就是正则图。

顶点的度 (Degree)：一个顶点的度是指与它相连的边的条数。

正则度 (k)：如果图中每个顶点的度都是 k，那么这个图就称为 k-正则图（k-regular graph）。